

Relatório de Auditoria de Segurança

Revisão de código em busca de cinco categorias de falhas de segurança

Data: 31 de agosto de 2026

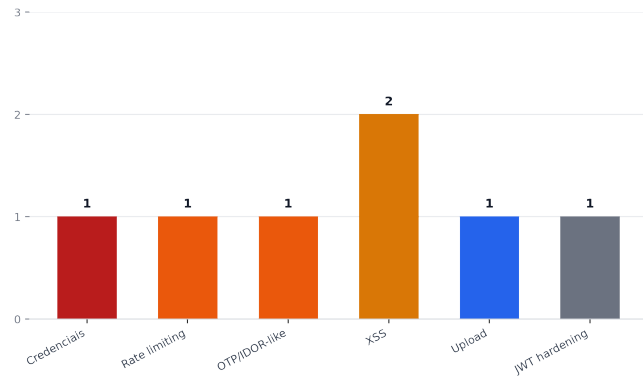
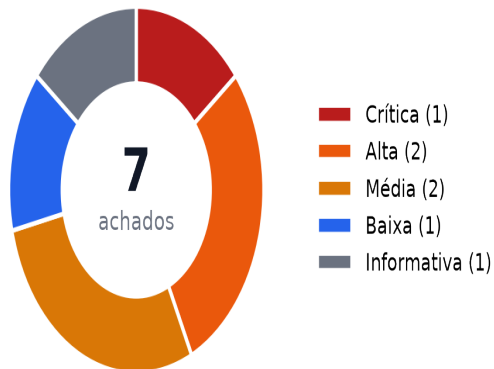
Escopo: backend Fastify/Prisma, frontend TanStack Start, configuração de deploy (Vercel)

Repositório: Fontthini/america-anatomy

Nota metodológica

Stack detectada: backend Node.js + Fastify 5 + Prisma 6 (PostgreSQL/Supabase) com autenticação JWT própria (jsonwebtoken + argon2), validação via zod; frontend React + TanStack Start; deploy Vercel (backend como função serverless, frontend com preset "vercel" do Nitro), sem Docker/CI/Terraform no repositório. Mapeamento das categorias para esta stack: (1) isolamento de posse — não há RLS ativo no caminho da aplicação (Prisma conecta com a role privilegiada do Postgres, que ignora RLS); o mecanismo real é filtragem manual por assignedSalesRepId/userId em cada serviço, auditado handler por handler. (2) permissão no navegador — cada rota sensível foi cruzada com seu requireRole no backend. (3) IDOR — todos os 10 módulos de rotas do backend foram lidos por completo, não por amostragem. (4) chaves expostas — varredura no código-fonte, .env.example, seed e histórico completo do git. (5) XSS — varredura em dangerouslySetInnerHTML/innerHTML/href/src no frontend e nas templates de e-mail HTML do backend.

Resumo executivo

1 Crítica**2** Alta**2** Média**1** Baixa**1** Informativa

Total de 7 achados verificados no código real (nenhuma especulação), além de 11 pontos fortes confirmados que já protegem o sistema hoje. O achado mais urgente (F1) é imediatamente acionável: credenciais de demonstração com papel ADMIN estão ativas no mesmo banco agora atrás do deploy de produção.

Pontos fortes verificados

O que já está protegido corretamente, com evidência no código — prova da cobertura desta auditoria.

✓ Isolamento por papel no CRM

crm.service.ts

assertLeadAccess/updateLead/updateFunnelStage/claimLead checam explicitamente que um SALES_REP só acessa leads com assignedSalesRepld igual ao próprio ator — verificado em TODOS os handlers de :id do módulo CRM, não só numa amostra.

✓ IDOR em notificações

notifications.service.ts

markNotificationRead verifica notification.userId !== userId antes de marcar como lida, retornando 404 genérico em vez de vaziar a existência do registro de outro usuário.

✓ Pedidos sempre autoescopados

orders.service.ts

createOrder/listMyOrders resolvem o doctorProfileId a partir do req.user.id do token — o cliente nunca informa de quem é o pedido, eliminando IDOR por design.

✓ Materiais e roster de curso

catalog.service.ts

assertCourseAccess (staff OU instrutor responsável daquele curso específico) é chamada em TODOS os handlers de materiais/registrations/roster antes de qualquer leitura ou escrita — inclusive resolvendo o catalogItemId correto a partir do id do sub-recurso.

✓ Permissão de escrita no backend, não só na UI

*.routes.ts (10 módulos)

Toda rota de criação/edição/exclusão sensível (catálogo, financeiro, blog, banners, exclusão de médico) tem requireRole no preHandler do Fastify — não depende da UI esconder botões por papel.

✓ Nenhum segredo hardcoded

repositório completo + histórico
git

Busca por padrões de chave/segredo no código-fonte não encontrou nada; nenhum arquivo .env real foi commitado em nenhum momento do histórico do git.

✓ Validação de ambiente que falha ao iniciar

backend/src/config/env.ts

JWT_ACCESS_SECRET, JWT_REFRESH_SECRET e COOKIE_SECRET são obrigatórios com mínimo de 32 caracteres via zod — sem valor padrão inseguro. O servidor chama process.exit(1) e recusa subir se estiverem ausentes ou curtos demais.

✓ Frontend sem segredo embutido

frontend/src/lib

Única variável VITE_* usada é VITE_API_URL (uma URL pública por natureza); nenhum arquivo .env é versionado no frontend.

✓ Sem HTML perigoso no React

frontend/src (varredura completa)

Único uso de dangerouslySetInnerHTML é um script estático de tema (sem dado de usuário); conteúdo de artigo é sempre renderizado como texto JSX puro (auto-escapado); o ID do vídeo do YouTube é extraído com regex restritiva ([w-]{6,}), sem chance de quebrar o atributo src.

✓ Cadastro nunca aceita role do cliente

auth.schemas.ts /
doctors.schemas.ts

Nenhum schema de registro (público ou de médico) aceita o campo role — o servidor sempre define DOCTOR por padrão, eliminando auto-elevação de privilégio no cadastro.

✓ Middlewares de autorização falham fechado

authenticate.ts / require-role.ts /
require-approved.ts

Token ausente, inválido ou expirado sempre resulta em 401; papel fora da lista permitida sempre resulta em 403 — nenhum caminho identificado que permita passagem por erro ou exceção não tratada.

Pontos fracos e achados detalhados

Cada achado foi verificado diretamente no código, com arquivo e linha exatos.

CRÍTICA**F1 · Chaves/Credenciais expostas**

Credenciais admin de demonstração ativas no banco agora em produção

backend/prisma/seed.ts:4-8, 28

O seed cria contas com senha fraca e documentada no próprio código-fonte (admin@demo.com / demo1234, role ADMIN; gerente@demo.com, vendedor@demo.com, medico@demo.com, pendente@demo.com — todas com a mesma senha). Esse é o MESMO banco Supabase que agora está atrás do backend publicado em produção na Vercel (america-anatomy-backend.vercel.app). Qualquer pessoa que leia o repositório público — ou tente credenciais padrão comuns — consegue logar como ADMIN.

Por que é explorável: Acesso ADMIN completo: gestão de médicos, contatos do CRM, financeiro e catálogo. É o achado de maior impacto porque não depende de nenhuma outra falha — é login direto, documentado, e a conta já existe no ambiente real.

```
/*
 * Admin: admin@demo.com / demo1234 (role ADMIN)
 * ...
 */
const passwordHash = await hash("demo1234");
```

ALTA**F2 · Autenticação / Rate limiting**

Nenhum rate limiting nos endpoints de autenticação

backend/src/modules/auth/auth.routes.ts:17-26

/api/auth/login, /api/auth/confirm-email, /api/auth/forgot-password e /api/auth/resend-confirmation não têm nenhum controle de taxa (nenhuma lib de rate limit está instalada no projeto — confirmado em package.json). Nada impede tentativas automatizadas e paralelas de login ou de adivinhação de código de verificação.

Por que é explorável: Combinado ao achado F1 (senhas fracas conhecidas) e F3 (OTP de 6 dígitos), a ausência de rate limit torna o brute force prático, especialmente hospedado em funções serverless onde a concorrência é barata para um atacante.

```
app.post("/api/auth/login", handleLogin);
app.post("/api/auth/confirm-email", handleConfirmEmail);
```

ALTA**F3 · Autenticação / IDOR-like**

Código de confirmação de e-mail não é vinculado a um usuário específico

backend/src/modules/auth/auth.service.ts:218-244

confirmEmail(input) recebe só { code }. O código de 6 dígitos (1.000.000 de combinações, gerado por generateOtpCode em backend/src/lib/token.ts) é buscado no banco só pelo hash do código — não é exigido e-mail nem qualquer vínculo com a sessão que originou o cadastro. Sem rate limit (F2), um código válido de QUALQUER usuário pendente pode ser encontrado por tentativa e erro.

Por que é explorável: Permite marcar o e-mail de outro usuário como verificado sem provar posse dele — quebra a garantia central de um fluxo de verificação de e-mail.

```
const tokenHash = hashToken(input.code);
const record = await prisma.verificationToken.findUnique({ where: { tokenHash } });
// nenhuma checagem de email/userId do chamador
```

MÉDIA**F4 · Inputs sem tratamento (XSS)**

URL de material de curso/artigo aceita esquema javascript: sem validação

frontend/src/routes/_medico.medico.cursos.\$id.tsx:157; _app.gestao-cursos.\$id.tsx:205; _medico.medico.blog.\$id.tsx:92

createCourseMaterialSchema usa z.string().url() (catalog.schemas.ts), que aceita qualquer URL sintaticamente válida — incluindo "javascript:alert(document.cookie)". O valor vai direto para href={m.url} num . Materiais de curso podem ser criados por MANAGER/ADMIN ou pelo médico definido como instrutor responsável do curso (assertCourseAccess em catalog.service.ts) — não só staff de confiança máxima.

Por que é explorável: Um instrutor (conta DOCTOR) mal-intencionado ou comprometido pode injetar um link que executa script no navegador de outro médico que clicar nele, dentro da sessão autenticada dessa vítima (token de acesso acessível via localStorage).

```
<a href={m.url} target="_blank" rel="noreferrer">
  {m.title}
</a>
```

MÉDIA

F5 · Inputs sem tratamento (XSS)

Nome do usuário e motivo de rejeição interpolados sem escape no HTML do e-mail*backend/src/lib/email/templates.ts (múltiplas funções)*

confirmCodeTemplate, resetPasswordTemplate, doctorPendingApprovalTemplate, doctorApprovedTemplate e doctorRejectedTemplate colocam `${data.name}` (e `${data.reason}`, digitado por um MANAGER/ADMIN ao rejeitar um médico) direto numa template string HTML, sem nenhuma função de escape. Não há lib de sanitização (dompurify, sanitize-html, etc.) no projeto. name vem de registerSchema/doctorRegisterSchema, que só valida tamanho — nenhuma restrição de caracteres.

Por que é explorável: É possível registrar uma conta com nome contendo HTML/JS e ele será embutido cru no e-mail. O caso mais claro de impacto cruzado é reason (staff → médico rejeitado): um HTML malicioso digitado no motivo de rejeição chega no e-mail do médico. Mesmo sem execução de script (clientes de e-mail modernos costumam bloquear `<script>`), a injeção de HTML/CSS permite phishing (trocar o texto de um link, esconder avisos).

```
<h1>Bem-vindo, ${data.name}!</h1>
...
${data.reason ? `<p><strong>Motivo:</strong> ${data.reason}</p>` : ""}
```

BAIXA

F6 · Inputs sem tratamento / Upload

Validação de tipo de arquivo do avatar confia no Content-Type declarado pelo cliente*backend/src/modules/users/users.controller.ts:15-32*

handleUploadAvatar valida data.mimetype (cabeçalho enviado pelo próprio cliente no multipart), não o conteúdo real do arquivo. Um arquivo com bytes de outro formato pode ser enviado se o cliente declarar Content-Type: image/png. O impacto direto é limitado hoje porque avatarUrl é sempre renderizado via `<img src=...>` em todo o frontend (nunca como iframe/HTML), o que não executa conteúdo malicioso — mas é um padrão de validação inseguro (CWE-434) que pode virar um vetor de verdade se o arquivo passar a ser servido/aberto de outra forma no futuro.

Por que é explorável: Confiar em Content-Type declarado pelo cliente é uma validação que pode ser contornada — o correto é inspecionar os bytes reais (magic number) do arquivo.

```
const mimeType = data.mimetype as string;
if (!ALLOWED_AVATAR_TYPES.includes(mimeType as ...)) { throw ... }
// mimeType vem do cabeçalho enviado pelo cliente, não do conteúdo do arquivo
```

INFORMATIVA F7 · Autenticação / Hardening**jwt.verify() não fixa a lista de algoritmos aceitos***backend/src/lib/jwt.ts:26-28*

verifyAccessToken chama `jwt.verify(token, env.JWT_ACCESS_SECRET)` sem o parâmetro `algorithms`. Não é explorável hoje porque o projeto não usa chaves assimétricas (RS256) em nenhum lugar — não há chave pública para o clássico ataque de confusão de algoritmo. Ainda assim, é uma boa prática de defesa em profundidade fixar `algorithms: ["HS256"]` explicitamente, para que o comportamento fique correto por design e não por acidente de configuração atual.

Por que é explorável: Reduz a superfície de erro futuro caso o projeto evolua para aceitar chaves assimétricas ou outra biblioteca com comportamento padrão diferente.

```
export function verifyAccessToken(token: string): AccessTokenPayload {  
  return jwt.verify(token, env.JWT_ACCESS_SECRET) as AccessTokenPayload;  
}
```

Recomendações priorizadas

Prior.	Recomendação	Detalhe
P1	Trocar/desativar as contas de demonstração no banco de produção agora (F1)	Rotacionar a senha de admin@demo.com e das demais contas *@demo.com no banco em uso pela Vercel, ou desativá-las, antes de divulgar a URL de produção.
P1	Adicionar rate limiting nos endpoints de autenticação (F2)	@fastify/rate-limit (ou equivalente) em /api/auth/login, /confirm-email, /forgot-password e /resend-confirmation, por IP e por conta.
P1	Vincular o código de confirmação de e-mail à sessão/usuário (F3)	Exigir o e-mail junto com o código em confirmEmailSchema, ou usar um token de alta entropia como no fluxo de redefinição de senha.
P2	Bloquear esquemas perigosos em URLs de material (F4)	Validar com uma allowlist de protocolo (http/https) em vez de z.string().url() puro, tanto em createCourseMaterialSchema quanto no schema de materiais de artigo do blog.
P2	Escapar variáveis antes de interpolar em HTML de e-mail (F5)	Aplicar uma função de escape simples (ou uma lib como escape-html) em todo dado dinâmico usado dentro de baseTemplate/confirmCodeTemplate/etc.
P3	Validar o conteúdo real do arquivo de avatar, não só o Content-Type (F6)	Checar magic bytes do buffer recebido (ex.: file-type) além do mimetype declarado.
P3	Fixar algorithms: ["HS256"] no jwt.verify() (F7)	Ajuste de dureza defensiva, sem exploração conhecida no estado atual do projeto.

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar na criação de cada issue.

--- ISSUE 1 ---

[Segurança] Credenciais admin de demonstração ativas no banco de produção

Labels: security, critical

Problema

O seed (`backend/prisma/seed.ts`) cria contas com senhas fracas e documentadas no próprio código-fonte, incluindo `admin@demo.com` / `demo1234` com papel `ADMIN`. Esse é o mesmo banco Supabase usado pelo backend agora publicado em produção (`america-anatomy-backend.vercel.app`).

Por que é explorável

Qualquer pessoa com acesso ao repositório (ou que tente credenciais padrão comuns) consegue logar como ADMIN em produção – acesso completo a CRM, financeiro e catálogo.

Evidência

```
`backend/prisma/seed.ts:4-8, 28`  
``ts
```

```
// * Admin: admin@demo.com / demo1234 (role ADMIN)  
const passwordHash = await hash("demo1234");  
``
```

Impacto

Tomada de conta ADMIN completa no ambiente de produção.

Sugestão de correção

Rotacionar/desativar todas as contas `\*@demo.com` no banco de produção, ou usar um processo de seed separado para dev/prod que nunca crie credenciais fracas em prod.

Critérios de aceite

- [] Nenhuma conta com senha do seed de demonstração existe no banco de produção
- [] Login com `admin@demo.com` / `demo1234` falha em produção
- [] Processo de seed documentado para não repetir isso em deploys futuros

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Ausência de rate limiting nos endpoints de autenticação

Labels: security, high

Problema

Nenhum endpoint de autenticação (`/api/auth/login`, `/confirm-email`, `/forgot-password`, `/resend-confirmation`) tem controle de taxa. Nenhuma lib de rate limit está instalada no backend.

Por que é explorável

Habilita brute force de senha e de código de verificação sem qualquer fricção, especialmente barato em infraestrutura serverless.

Evidência

```
`backend/src/modules/auth/auth.routes.ts:17-26`  
``ts
```

```
app.post("/api/auth/login", handleLogin);  
app.post("/api/auth/confirm-email", handleConfirmEmail);  
``
```

Impacto

Brute force de credenciais e de códigos OTP.

Sugestão de correção

Adicionar `@fastify/rate-limit` (por IP e por conta) nas rotas de autenticação.

Critérios de aceite

- [] Requisições em excesso a `/api/auth/login` retornam 429 após N tentativas
- [] Mesmo controle aplicado a confirm-email/forgot-password/resend-confirmation
- [] Limite documentado (ex.: N tentativas por minuto por IP+conta)

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Código de confirmação de e-mail não vinculado a um usuário específico

Labels: security, high

Problema

`confirmEmail` recebe só `{ code }` (6 dígitos, 1.000.000 de combinações) e busca no banco só pelo hash do código, sem exigir e-mail/userId do chamador.

Por que é explorável

Sem rate limit (ver issue relacionada), um código válido de qualquer usuário pendente pode ser encontrado por tentativa e erro, permitindo marcar o e-mail de outra pessoa como verificado.

Evidência

```
`backend/src/modules/auth/auth.service.ts:218-224`  
``ts  
const tokenHash = hashToken(input.code);  
const record = await prisma.verificationToken.findUnique({ where: { tokenHash } });  
// nenhuma checagem de email/userId do chamador  
``
```

Impacto

Verificação de e-mail de outro usuário sem provar posse da caixa de entrada.

Sugestão de correção

Exigir o e-mail junto ao código em `confirmEmailSchema` e cruzar com `record.userId`, ou migrar para um token de alta entropia como no fluxo de redefinição de senha.

Critérios de aceite

- [] `POST /api/auth/confirm-email` exige e-mail + código
- [] Código válido de outro e-mail é rejeitado
- [] Teste automatizado cobre o caso de código correto com e-mail errado

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] URLs de material de curso/artigo aceitam esquema javascript:

Labels: security, medium

Problema

`createCourseMaterialSchema` usa `z.string().url()`, que aceita qualquer URL sintaticamente válida – incluindo `javascript:...`. O valor vai direto para `href={m.url}` num link clicável.

Por que é explorável

Materiais podem ser criados por staff OU pelo instrutor responsável do curso (conta `DOCTOR`) – não só por administradores. Um instrutor malicioso ou comprometido pode injetar um link que executa script na sessão de outro médico que clicar.

Evidência

```
`frontend/src/routes/_medico.medico.cursos.$id.tsx:157` (mesmo padrão em  
`_app.gestao-cursos.$id.tsx:205` e `_medico.medico.blog.$id.tsx:92`)  
``tsx  
<a href={m.url} target="_blank" rel="noreferrer">{m.title}</a>
```

```

...

## Impacto
XSS armazenado, executando na sessão autenticada de quem clicar no link.

## Sugestão de correção
Validar com allowlist de protocolo (`http:`/`https:`) em vez de `z.string().url()` puro, nos dois
schemas (curso e material de artigo).

## Critérios de aceite
- [ ] Tentar salvar uma URL `javascript:` retorna erro de validação
- [ ] Apenas `http://` e `https://` são aceitos nos dois formulários de material
- [ ] Teste automatizado cobre a rejeição do esquema `javascript:`
--- FIM ISSUE 4 ---

```

--- ISSUE 5 ---

[Segurança] Variáveis interpoladas sem escape no HTML dos e-mails transacionais

Labels: security, medium

```

## Problema
`backend/src/lib/email/templates.ts` interpola `${data.name}` e `${data.reason}` direto em strings
HTML, sem nenhuma função de escape ou lib de sanitização (nenhuma está instalada no projeto).

## Por que é explorável
`name` vem do cadastro do usuário (sem restrição de caracteres); `reason` é digitado por um
MANAGER/ADMIN ao rejeitar um médico e chega no e-mail do médico rejeitado – um caso claro de injeção
cruzada entre usuários, além de possibilitar phishing via HTML/CSS mesmo sem execução de script.

## Evidência
`backend/src/lib/email/templates.ts` (confirmCodeTemplate, resetPasswordTemplate,
doctorPendingApprovalTemplate, doctorApprovedTemplate, doctorRejectedTemplate)
```ts
<h1>Bem-vindo, ${data.name}</h1>
${data.reason ? `<p><strong>Motivo:</strong> ${data.reason}</p>` : ""}
```

## Impacto
Injeção de HTML nos e-mails enviados pelo sistema – risco de phishing e quebra de layout; execução de
script se o cliente de e-mail do destinatário não bloquear `<script>`.

## Sugestão de correção
Escapar todo dado dinâmico (`<`, `>`, `&`, `\"`, `\'`) antes de interpolar nas templates, ou usar uma
lib de escape/sanitização dedicada.

## Critérios de aceite
- [ ] Nome/motivo contendo `<`, `>`, `\"` aparece como texto literal no e-mail, não como HTML
- [ ] Cobertura aplicada a todas as 5 templates que interpolam dado de usuário
--- FIM ISSUE 5 ---

```

--- ISSUE 6 ---

[Segurança] Endurecimentos menores: validação de avatar e algoritmo do JWT

Labels: security, low

```

## Problema
Dois ajustes de dureza defensiva, sem exploração prática confirmada hoje:

1. **Upload de avatar confia no Content-Type declarado pelo cliente**, não no conteúdo real do
arquivo (`backend/src/modules/users/users.controller.ts:15-32`).
2. **`jwt.verify()` não fixa `algorithms`** explicitamente (`backend/src/lib/jwt.ts:26-28`) – não

```

explorável hoje pois o projeto não usa chaves assimétricas, mas é boa prática travar o algoritmo esperado.

Impacto

Baixo no estado atual (avatar sempre renderizado via `

Sugestão de correção

1. Verificar magic bytes do buffer recebido, não só `mimetype` declarado.
2. Passar `{ algorithms: ["HS256"] }` em `jwt.verify()`.

Critérios de aceite

- [] Upload com Content-Type falso mas conteúdo incompatível é rejeitado
- [] `jwt.verify` só aceita tokens assinados com HS256

--- FIM ISSUE 6 ---